

MediStock

MVC Pharmacy Inventory System

A complete beginner-friendly guide to understanding how this PHP MVC project works. Built from scratch with a custom framework — no Laravel, no CodeIgniter, just pure PHP.

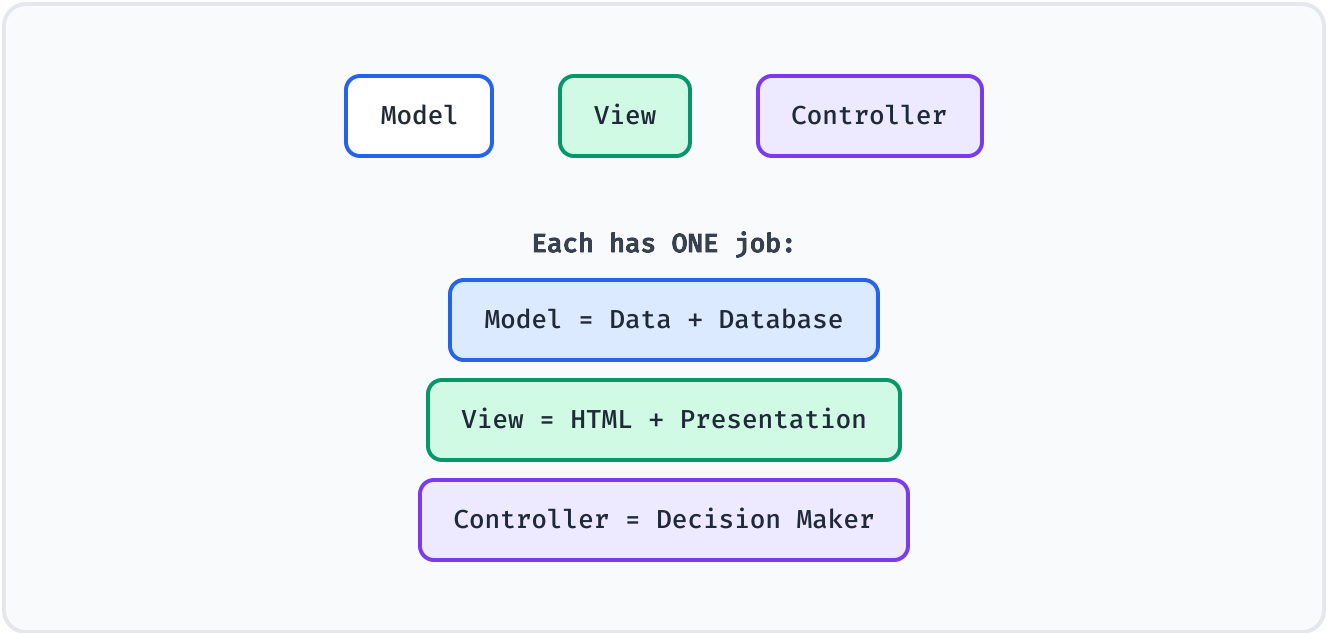
PHP 8 • MySQL • Custom MVC Framework • XAMPP

Table of Contents

1. What is MVC and Why Use It?
2. Project Overview — What Does MediStock Do?
3. Complete Directory Structure
4. How a Request Flows (End to End)
5. The Core Framework (core/)
6. The Entry Point (public/index.php)
7. Routing — How URLs Map to Code
8. Database Connection — Singleton Pattern
9. Controllers — The Business Logic
10. Models — Talking to the Database
11. Views — The HTML Templates
12. Middleware — Authentication & Security
13. Security Features
14. The Database Schema
15. Assets (CSS & JavaScript)
16. How to Create Your Own Project Like This

1. What is MVC and Why Use It?

MVC stands for **Model-View-Controller**. It is a design pattern that splits your application into three separate parts, each with a clear job:



Component	Job	Examples in This Project
Model	Reads/writes data from the database. Contains business rules about data.	Medicine.php, User.php, Sale.php
View	Shows HTML to the user. Never queries the database directly.	login.php, medicines/index.php, header.php
Controller	Receives user requests, asks Models for data, picks which View to show.	MedicineController.php, AuthController.php

Why MVC?

Without MVC, your code becomes a tangled mess of HTML mixed with database queries. MVC keeps things organized: you can change the database without touching the HTML, or redesign the UI without breaking business logic.

2. Project Overview — What Does MediStock Do?

MediStock is a **pharmacy medicine inventory management system**. It helps a pharmacy track medicines, record sales, manage stock, and generate reports.

Key Features

- **Login & User Management** — Admin and Staff roles with secure password hashing
- **Medicine CRUD** — Add, view, edit, and soft-delete medicines
- **Sales Recording** — Record sales with stock validation and transaction safety
- **Stock Adjustments** — Record damaged/expired/lost stock with audit trail
- **Reports** — Low stock alerts, expiry alerts, prescription audit
- **User Avatars** — Upload and serve profile pictures
- **Settings** — Configurable thresholds and pharmacy info

Default Login

Username	Password	Role
admin	admin123	Admin (full access)

3. Complete Directory Structure

```

mvc_project/ |— app/ ← YOUR APPLICATION CODE | |— Controllers/ ← Handle
user requests | | |— AuthController.php | | |— AvatarController.php | | |—
DashboardController.php | | |— MedicineController.php | | |—
ReportController.php | | |— SaleController.php | | |—
SettingsController.php | | |— StockAdjustmentController.php | | |—
UserController.php | |— Models/ ← Database operations | | |— Category.php |
| | |— Medicine.php | | |— PrescriptionDetail.php | | |— Sale.php | | |—
Setting.php | | |— StockAdjustment.php | | |— Supplier.php | | |— User.php
| |— Views/ ← HTML templates | |— auth/ (login page) | |— dashboard/ (main
dashboard) | |— errors/ (404 page) | |— layouts/ (header.php, footer.php) |
| |— medicines/ (CRUD views) | |— reports/ (report pages) | |— sales/ (sale
pages) | |— settings/ (settings page) | |— users/ (user management) | |—
assets/ ← Source CSS/JS files | |— css/style.css | |— js/script.js | |—
config/ ← Configuration files | |— database.php ← MySQL credentials | |—
core/ ← FRAMEWORK ENGINE | |— Controller.php ← Base controller (all
controllers inherit this) | |— Database.php ← Database connection
(Singleton) | |— Middleware.php ← Auth checks | |— Model.php ← Base model
(CRUD operations for free) | |— Router.php ← URL matching engine | |—
View.php ← Helper functions for views | |— database/ ← Database structure |
| |— schema.sql ← Complete DB schema + seed data | |— public/ ← WEBSITE ROOT
(Apache points here) | |— .htaccess ← URL rewriting rules | |— index.php ←
ENTRY POINT - everything starts here | |— assets/ ← Copy of CSS/JS for web
serving | |— routes/ ← URL-to-controller mappings | |— web.php ← All route
definitions | |— uploads/ ← User-uploaded files | |— avatars/ ← Profile
pictures | |— setup.sh ← XAMPP deployment script

```

4. How a Request Flows (End to End)

When a user types `http://localhost/medistock/public/medicines` in their browser, here is exactly what happens, step by step:

1

Apache Rewrites the URL

The `.htaccess` file says: "If the requested file doesn't physically exist, send it to `index.php` ." Since `/medicines` is not a real file, Apache redirects to `index.php` .

```
# public/.htaccess
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]
```

2

index.php Boots the Application

The entry point does 5 things in order:

1. Registers an **autoloader** (so PHP can find classes automatically)
2. Starts the **session**
3. Sets **security headers**
4. Loads **routes** from `routes/web.php`
5. **Dispatches** the request (matches URL to a route)

3

Router Matches the URL

The Router checks each registered route. It finds: `$router→get('/medicines', 'MedicineController@index')`. The URL matches, so it will call `MedicineController::index()`.

4

Controller Handles the Request

`MedicineController::index()` runs. It checks if the user is logged in (Middleware), asks the `Medicine` model for data, and loads the view file.

5

Model Queries the Database

The controller calls `$this→medicineModel→getFiltered()`. The Model builds a SQL query, executes it through the Database class, and returns an array of medicines.

6

View Renders the HTML

The controller does `require __DIR__ . '/../Views/medicines/index.php'`. The view file loops through the medicines array and outputs HTML. The header and footer layouts are included automatically.

7

Response Sent to Browser

PHP sends the complete HTML page back to the user's browser. The user sees the medicines list.

Think of it like a restaurant:

The

Router

is the host who seats you. The

Controller

is the waiter who takes your order. The

Model

is the kitchen that prepares the food. The

View

is the plate that presents the food to you.

5. The Core Framework (core/)

This project does NOT use Laravel, CodeIgniter, or any framework. The developer built a **mini-framework from scratch** in 6 files. This is the engine that powers everything.

File	What It Does	Lines
<code>Router.php</code>	Matches URLs to controller methods using regex	125
<code>Database.php</code>	Singleton MySQL connection with prepared statements	157
<code>Controller.php</code>	Base class: view rendering, redirects, CSRF, flash messages	118
<code>Model.php</code>	Base class: CRUD operations (find, all, create, update, delete)	150
<code>Middleware.php</code>	Auth/role checks (auth, admin, guest)	52
<code>View.php</code>	Helper: alerts, CSRF field, URL generation, escaping	73

Key Insight:

The `core/` folder has ZERO business logic. It is pure infrastructure. You could reuse this exact same `core/` folder to build a completely different application (like a blog or e-commerce site) without changing a single line in it.

6. The Entry Point (public/index.php)

Every request goes through one file: `public/index.php`. This is called the **Front Controller Pattern**.

```
// public/index.php (simplified)

// 1. AUTOLOADER - Maps class names to file paths
spl_autoload_register(function (string $class) {
    $prefixes = [
```

```

        'App\\' => __DIR__ . '/../app/',    // App\Controllers\* → app/Controller
        'Core\\' => __DIR__ . '/../core/',  // Core\Router → core/Router.php
    ];
    // ... finds the file and requires it
});

// 2. SESSION
session_start();

// 3. SECURITY HEADERS
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');

// 4. BASE URL
define('BASE_URL', rtrim(dirname($_SERVER['SCRIPT_NAME']), '/'));

// 5. LOAD ROUTES
require __DIR__ . '/../routes/web.php'; // Creates $router with all routes

// 6. DISPATCH
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$router->dispatch($_SERVER['REQUEST_METHOD'], $uri);

```

How the Autoloader Works

Without an autoloader, you'd need `require 'Router.php'` at the top of every file. The autoloader says: "When PHP sees `Core\Router`, look for `core/Router.php` and load it automatically." This is a simplified version of PSR-4, the PHP standard for autoloading.

7. Routing — How URLs Map to Code

Defining Routes (routes/web.php)

```
// routes/web.php
$route = new \Core\Router();

$route->get('/medicines',          'MedicineController@index');
$route->get('/medicines/{id}',     'MedicineController@show');
$route->post('/medicines/store',   'MedicineController@store');
```

This creates 3 routes:

HTTP Method	URL	Controller Method	Purpose
GET	/medicines	MedicineController::index()	List all medicines
GET	/medicines/5	MedicineController::show(5)	View medicine #5
POST	/medicines/store	MedicineController::store()	Save a new medicine

How Parameters Work

When the Router sees `/medicines/{id}`, it converts it to a regex:

```
// Router::buildPattern() converts:
// /medicines/{id}
// to:
// #^/medicines/(?P<id>[^/]+)$#

// Then when /medicines/5 is requested:
// preg_match extracts: $matches['id'] = '5'
// Calls: MedicineController->show('5')
```

Route Types

- `$router→get( ... )` — Only matches GET requests (browser navigating to a page)
- `$router→post( ... )` — Only matches POST requests (form submissions)
- `$router→any( ... )` — Matches both GET and POST

8. Database Connection — Singleton Pattern

The Database class uses the **Singleton Pattern** — only ONE database connection can ever exist during a request.

```
// core/Database.php (simplified)
class Database
{
    private static ?self $instance = null; // The one instance
    private \mysqli $connection;

    private function __construct() // Private = can't do "new Database()"
    {
        $config = require __DIR__ . '/../config/database.php';
        $this->connection = new \mysqli(
            $config['host'],      // localhost
            $config['username'],  // root
            $config['password'],  // (empty)
            $config['database']   // medistock_mvc
        );
    }

    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self(); // Create only once
        }
        return self::$instance;
    }

    public function query(string $sql, array $params = [], string $types = ''): mixed
    {
        $stmt = $this->connection->prepare($sql);
        $stmt->bind_param($types, ... $params); // Safe from SQL injection!
        $stmt->execute();
        return $stmt->get_result();
    }
}
```

Why Singleton?

Without Singleton, every Model would create its own database connection. With 10 queries, you'd have 10 connections. Singleton ensures all Models share ONE connection, which is more efficient and prevents connection limits from being hit.

Database Config (config/database.php)

```
return [  
    'host'      ⇒ 'localhost',  
    'username' ⇒ 'root',  
    'password' ⇒ '',  
    'database' ⇒ 'medistock_mvc',  
];
```

9. Controllers — The Business Logic

Controllers are the "middlemen." They receive user input, call Models for data, and load Views for output.

Example: MedicineController (simplified)

```
class MedicineController extends Controller
{
    private Medicine $medicineModel;

    public function __construct()
    {
        parent::__construct();
        $this->medicineModel = new Medicine(); // Create the model
    }

    // GET /medicines - List all medicines
    public function index(): void
    {
        Middleware::auth(); // Must be logged in

        $search = $this->query('search', '');
        $medicines = $this->medicineModel->getFiltered($search); // Ask model for data

        require __DIR__ . '/../Views/medicines/index.php'; // Load the view
    }

    // POST /medicines/store - Save a new medicine
    public function store(): void
    {
        Middleware::auth();

        // 1. Validate CSRF token
        if (!$this->validateCsrf($this->input('csrf_token'))) {
            $this->setFlash('danger', 'Invalid request.');
```

```
            $this->redirect('/medicines/create');
```

```
        }

        // 2. Get and validate input
```

```

$name = trim($this->input('name'));
if (empty($name)) {
    $this->setFlash('danger', 'Name is required.');
```

```

    $this->redirect('/medicines/create');
}

// 3. Save to database via model
$id = $this->medicineModel->create([
    'name' => $name,
    'quantity' => (int) $this->input('quantity'),
    // ... more fields
]);

// 4. Set success message and redirect
$this->setFlash('success', "Medicine added! (ID: {$id})");
$this->redirect('/medicines');
}
}
```

The PRG Pattern (Post-Redirect-Get)

PRG Pattern:

After saving data via POST, always redirect to a GET page. This prevents duplicate form submissions if the user refreshes. Notice how `store()` ends with `$this->redirect('/medicines')`.

Base Controller Features

Every controller inherits these from `Core\Controller`:

Method	What It Does
<code>\$this->view('name')</code>	Renders a view file with data
<code>\$this->redirect('/path')</code>	HTTP redirect (with BASE_URL prefix)
<code>\$this->setFlash('success', 'msg')</code>	Stores a one-time message in session
<code>\$this->getFlash('success')</code>	Reads and deletes a flash message

<code>\$this->csrfToken()</code>	Generates/returns CSRF token
<code>\$this->validateCsr(\$token)</code>	Validates CSRF token (timing-safe)
<code>\$this->isPost()</code>	Checks if request is POST
<code>\$this->input('key')</code>	Reads POST value
<code>\$this->query('key')</code>	Reads GET value

10. Models — Talking to the Database

Models handle ALL database operations. They never output HTML. They return arrays and integers.

Base Model (core/Model.php)

The base Model gives you CRUD operations for free. Every model inherits these:

```
// You get these methods automatically:
$model→find(5);           // SELECT * FROM table WHERE id = 5
$model→all();             // SELECT * FROM table
$model→all(['is_active' ⇒ 1]); // SELECT * FROM table WHERE is_active = 1
$model→create(['name' ⇒ 'X']); // INSERT INTO table (name) VALUES ('X')
$model→update(5, ['name' ⇒ 'Y']); // UPDATE table SET name='Y' WHERE id=5
$model→delete(5);         // DELETE FROM table WHERE id = 5
$model→count();           // SELECT COUNT(*) FROM table
$model→exists(['name' ⇒ 'X']); // Returns true/false
```

Custom Model: Medicine.php

```
class Medicine extends Model
{
    protected string $table = 'medicines'; // Which table?

    // Custom method: get medicines with category names, with search
    public function getFiltered(string $search, int $categoryId, int $limit, int $offset)
    {
        $sql = "SELECT m.*, c.name AS category_name
                FROM medicines m
                JOIN categories c ON m.category_id = c.id
                WHERE m.is_active = 1
                AND m.name LIKE ?           -- prepared statement, safe!
                ORDER BY m.created_at DESC
                LIMIT ? OFFSET ?";

        return $this→db→fetchAll($sql, ['%' . $search . '%', $limit, $offset],
        }
    }
}
```

```
// Custom: decrease stock atomically
public function decreaseStock(int $id, int $quantity): bool
{
    $this->db->query(
        "UPDATE medicines SET quantity = quantity - ?
        WHERE id = ? AND quantity ≥ ?",    -- prevents negative stock
        [$quantity, $id, $quantity],
        'iii'
    );
    return $this->db->affectedRows() > 0;
}
```

SQL Injection Prevention:

Notice every query uses `?` placeholders and `bind_param()`. User input is NEVER concatenated directly into SQL strings. This makes SQL injection impossible.



11. Views — The HTML Templates

Views are plain PHP files that output HTML. They receive data from Controllers via variables.

Layout System

Every page shares a common header and footer:

```

<!-- views/medicines/index.php -->
<?php
$pageTitle = 'All Medicines';
require __DIR__ . '/../layouts/header.php'; // Opens <html>, <head>, sidebar
?>

<!-- YOUR PAGE CONTENT GOES HERE -->
<table>
    <?php foreach ($medicines as $med): ?>
        <tr>
            <td><?= htmlspecialchars($med['name']) ?></td>
        </tr>
    <?php endforeach; ?>
</table>

<?php require __DIR__ . '/../layouts/footer.php'; ?> <!-- Closes HTML -->

```

Key View Helpers

Helper	What It Does
<code>\Core\View::url('/path')</code>	Generates full URL with BASE_URL prefix
<code>\Core\View::alert('success', \$msg)</code>	Renders a styled alert box
<code>\Core\View::csrfField()</code>	Outputs a hidden CSRF token input
<code>\Core\View::currentPage()</code>	Returns current page name (for menu highlighting)

`htmlspecialchars($var)`

Escapes HTML to prevent XSS attacks

Security Rule:

ALWAYS use `htmlspecialchars()` when outputting user-provided data. This prevents Cross-Site Scripting (XSS) attacks where attackers inject malicious JavaScript into your pages.

12. Middleware — Authentication & Security

Middleware checks run BEFORE a controller method executes. They protect pages from unauthorized access.

```
// core/Middleware.php
class Middleware
{
    // Must be logged in
    public static function auth(): void
    {
        if (!isset($_SESSION['user_id'])) {
            header('Location: /login');
            exit;
        }
    }

    // Must have a specific role
    public static function role(string $requiredRole): void
    {
        self::auth(); // First check login
        if (($_SESSION['role'] ?? '') !== $requiredRole) {
            header('Location: /dashboard');
            exit;
        }
    }

    // Must be admin
    public static function admin(): void
    {
        self::role('admin');
    }

    // Must be logged OUT (for login page)
    public static function guest(): void
    {
        if (isset($_SESSION['user_id'])) {
            header('Location: /dashboard');
            exit;
        }
    }
}
```

```
}  
}
```

How Controllers Use Middleware

```
// In any controller method:  
public function index(): void  
{  
    Middleware::auth();      // Redirect to /login if not logged in  
    // ... show page  
}  
  
public function store(): void  
{  
    Middleware::admin();      // Must be admin  
    // ... delete user  
}  
  
public function showLogin(): void  
{  
    Middleware::guest();      // Redirect to /dashboard if already logged in  
    // ... show login form  
}
```

13. Security Features

This project implements multiple security layers:

Feature	How It Works	File
CSRF Protection	Every form includes a hidden token. Server validates it on submission. Prevents cross-site request forgery.	Controller.php:79-93
SQL Injection Prevention	All queries use prepared statements with ? placeholders. User input never touches SQL directly.	Database.php:57-68

XSS Prevention	All output is escaped with <code>htmlspecialchars()</code> . Scripts injected into data cannot execute.	View.php:64-67
Password Hashing	Passwords stored as bcrypt hashes (<code>password_hash()</code>). Never stored in plain text.	User.php:37
Account Lockout	After 5 failed login attempts, account locks for 15 minutes.	User.php:25-50
Role-Based Access	Middleware checks user role (admin/staff) before allowing actions.	Middleware.php
Security Headers	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy.	public/index.php:33-36
Directory Traversal Prevention	Avatar controller uses <code>basename()</code> to strip path components.	AvatarController.php
Soft Deletes	Medicines are deactivated (<code>is_active = 0</code>), not permanently deleted.	MedicineController.php:234

14. The Database Schema

The project uses MySQL with 8 tables. Here is the structure:

```
medistock_mvc database
├── users                -- Login credentials, roles (admin/staff)
├── categories           -- Medicine types (Tablet, Syrup, etc.)
├── suppliers            -- Pharmaceutical suppliers
├── medicines            -- Core inventory (name, batch, expiry, qty, price)
│   ├── FK → categories.id
│   └── FK → suppliers.id
├── sales                -- Sale records (INV-YYYYMMDD-NNNN format)
│   └── FK → medicines.id
├── prescription_details -- Doctor/prescription info for controlled drugs
│   └── FK → sales.id (CASCADE delete)
├── stock_adjustments    -- Audit trail for stock changes
│   └── FK → medicines.id
├── settings             -- Key-value config (thresholds, pharmacy name)
└── audit_log            -- Security/compliance activity log
```

Key Relationships

- Each **medicine** belongs to one category and one supplier (Foreign Keys)
- Each **sale** links to one medicine
- **Prescription details** cascade-delete with their sale (if sale is deleted, prescription is too)
- **Stock adjustments** are permanent records for audit trail
- **Foreign key constraints** prevent orphaned records (e.g., can't delete a category that has medicines)

Seed Data

The schema includes sample data for testing: 8 categories, 4 suppliers, 10 medicines, 1 admin user (admin/admin123), and 5 settings.

15. Assets (CSS & JavaScript)

CSS (assets/css/style.css — 622 lines)

- CSS custom properties for consistent colors and spacing
- Fixed sidebar layout (260px) with dark background
- Responsive: sidebar collapses on mobile (<1024px)
- Component styles: cards, tables, forms, buttons, badges, alerts, pagination
- Print styles: hides sidebar/nav for clean printing
- Role-based CSS: `.role-admin` / `.role-staff` controls visibility

JavaScript (assets/js/script.js — 339 lines)

- Sidebar toggle for mobile
- Form submit loading states (spinner + disabled button)
- Client-side form validation (sale, medicine, user, adjustment forms)
- Delete confirmation dialogs
- Auto-dismiss success alerts after 5 seconds
- Character counter for textareas

16. How to Create Your Own Project Like This

Here is a step-by-step guide to building your own MVC project from scratch:

Step 1: Set Up Your Environment

1

Install XAMPP (or Laragon for Windows). It includes Apache, MySQL, and PHP.

```
# Verify PHP and MySQL are running
php -v
mysql --version
```

Step 2: Create the Directory Structure

2

```
mkdir my_project
cd my_project
mkdir -p app/Controllers app/Models app/Views/layouts
mkdir -p core config database public/assets/css public/assets/js routes upl
```

Step 3: Build the Core Framework

3

Start with these files in this order:

1. **config/database.php** — Your MySQL credentials
2. **core/Database.php** — Copy from this project (Singleton connection)

3. **core/Model.php** — Copy the base CRUD class
4. **core/Router.php** — Copy the URL matching engine
5. **core/Controller.php** — Copy the base controller
6. **core/View.php** — Copy the helper functions
7. **core/Middleware.php** — Copy the auth checks

Step 4: Create the Entry Point

4

Create `public/index.php` and `public/.htaccess` exactly as in this project. These never change.

Step 5: Create the Database

5

Design your tables in MySQL. Start simple:

```
-- Start with just what you need
CREATE TABLE users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'user') DEFAULT 'user'
);
```

Step 6: Create Your First Model

6

```
// app/Models/User.php
class User extends Model
{
```

```
protected string $table = 'users';

// Add custom methods as needed
public function findByUsername(string $username): ?array
{
    return $this->db->fetchOne(
        "SELECT * FROM users WHERE username = ?",
        [$username], 's'
    );
}
```

Step 7: Create Your First Controller

7

```
// app/Controllers/UserController.php
class UserController extends Controller
{
    public function index(): void
    {
        Middleware::auth();
        $users = (new User())->all();
        require __DIR__ . '/../Views/users/index.php';
    }
}
```

Step 8: Create Your First View

8

```
<!-- app/Views/users/index.php -->
<?php $pageTitle = 'Users'; ?>
<?php require __DIR__ . '/../layouts/header.php'; ?>

<h1>All Users</h1>
```

```
<table>
  <?php foreach ($users as $user): ?>
    <tr>
      <td><?= htmlspecialchars($user['username']) ?></td>
    </tr>
  <?php endforeach; ?>
</table>

<?php require __DIR__ . '/../layouts/footer.php'; ?>
```

Step 9: Define Your Routes

9

```
// routes/web.php
$router = new \Core\Router();
$router->get('/users', 'UserController@index');
$router->post('/users/store', 'UserController@store');
```

Step 10: Test and Iterate

10

Start Apache and MySQL in XAMPP, create the database, and visit

http://localhost/my_project/public/. Add features one at a time: search, pagination, authentication, file uploads.

The Golden Rule:

Keep the `core/` folder untouched. Add new features by creating new Models, Controllers, and Views. The framework handles the plumbing; you handle the business logic.

Quick Reference: Common Tasks

I want to...	What to do
Add a new page	1. Add route in <code>routes/web.php</code> 2. Create Controller method in <code>app/Controllers/</code> 3. Create View in <code>app/Views/</code>
Add a new database table	Add <code>CREATE TABLE</code> in <code>database/schema.sql</code> , create Model in <code>app/Models/</code>
Add login protection	Add <code>Middleware::auth();</code> as first line in controller method
Add admin-only protection	Add <code>Middleware::admin();</code> as first line
Add a new CSS style	Edit <code>assets/css/style.css</code> and <code>public/assets/css/style.css</code> (keep them identical)
Add a new JavaScript feature	Edit <code>assets/js/script.js</code> and <code>public/assets/js/script.js</code>
Add a new form	Create View with <code><form method="POST"></code> , add POST route, add CSRF field with <code>\Core\View::csrfField()</code>
Add search functionality	Add <code>WHERE ... LIKE ?</code> query in Model, pass <code>\$_GET['search']</code> from Controller

Key Takeaway

This project demonstrates that you don't need a big framework to build a real application. With ~1,500 lines of framework code in `core/` and ~3,000 lines of application code, you can build a complete, secure, production-ready system. The MVC pattern is the foundation — once you understand it, you can build anything.